Constraint-Based Problem Solving

Solutions — Questions 1, 2 and 3

Question 1 Solution: Backtracking Search Trace

The search proceeds depth-first, assigning one doctor at a time and checking constraints as soon as enough variables are bound (forward-checking style constraint propagation combined with backtracking).

Step	Variable Assigned	Domain Considered	Constraint Check	Result
1	D1 = Morning	{Morning, Afternoon} (Night removed by constraint i)	(i) D1 ≠ Night → satisfied	Continue
2	D2 = Morning	{Morning, Afternoon, Night}	(iii) Morning already taken by D1 → conflict	Fail, try next value
3	D2 = Afternoon	remaining {Afternoon, Night}	(iii) Afternoon free → no clash	Continue
4	D3 = Night	{Afternoon, Night} minus Morning(iv), minus taken → {Night}	(iv) D3 ≠ Morning ✓ (ii) D2(Afternoon) < D3(Night) ✓ (iii) all distinct ✓	Success — Goal Test Passed
5	D2 = Night (alt. branch)	{Afternoon, Night}	(ii) D2(Night) < D3 fails for the only remaining value {Afternoon}, since Night is after Afternoon	Fail → Backtrack
6	Backtrack to D1 = Afternoon (alt. branch)	D1 ∈ {Morning, Afternoon}	D2 = Morning, D3 = Night satisfies (ii),(iii),(iv)	Alternate valid solution found: D1=Afternoon, D2=Morning, D3=Night

Final Valid Schedule

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

Verification against all constraints:

- (i) D1 = Morning ≠ Night → satisfied

- (ii) D2 (Afternoon) is before D3 (Night) → satisfied
- (iii) Morning, Afternoon, Night are all distinct → satisfied
- (iv) D3 = Night ≠ Morning → satisfied
- (v) All three doctors are each assigned exactly one shift → satisfied

Final Answer: D1 → Morning, D2 → Afternoon, D3 → Night.

Note: an alternate valid solution (D1=Afternoon, D2=Morning, D3=Night) also exists, discovered while exploring the alternate branch of D1 — confirming this CSP has more than one solution.

Question 2 Solution: BFS Step-by-Step Trace

Step	Node Dequeued	Neighbours Generated (R,D,L,U order)	Newly Added to Queue	Queue After Step
1	(0,0) S	R(0,1) ok, D(1,0) ok, L/U out of bounds	(0,1),(1,0)	[(0,1),(1,0)]
2	(0,1)	R(0,2) ok, D(1,1) OBSTACLE, L(0,0) visited	(0,2)	[(1,0),(0,2)]
3	(1,0)	R(1,1) OBSTACLE, D(2,0) ok, U(0,0) visited	(2,0)	[(0,2),(2,0)]
4	(0,2)	R(0,3) ok, D(1,2) ok, L(0,1) visited	(0,3),(1,2)	[(2,0),(0,3),(1,2)]
5	(2,0)	R(2,1) OBSTACLE, D(3,0) ok, U(1,0) visited	(3,0)	[(0,3),(1,2),(3,0)]
6	(0,3)	R(0,4) ok, D(1,3) OBSTACLE, L(0,2) visited	(0,4)	[(1,2),(3,0),(0,4)]
7	(1,2)	R(1,3) OBSTACLE, D(2,2) ok, L(1,1) OBSTACLE, U(0,2) visited	(2,2)	[(3,0),(0,4),(2,2)]
8	(3,0)	R(3,1) OBSTACLE, D(4,0) ok, U(2,0) visited	(4,0)	[(0,4),(2,2),(4,0)]
9	(0,4)	D(1,4) ok, L(0,3) visited (R/U out of bounds)	(1,4)	[(2,2),(4,0),(1,4)]
10	(2,2)	R(2,3) ok, D(3,2) ok, L(2,1) OBSTACLE, U(1,2) visited	(2,3),(3,2)	[(4,0),(1,4),(2,3),(3,2)]
11	(4,0)	R(4,1) ok, U(3,0) visited (D out of bounds)	(4,1)	[(1,4),(2,3),(3,2),(4,1)]
12	(1,4)	D(2,4) ok, L(1,3) OBSTACLE, U(0,4) visited	(2,4)	[(2,3),(3,2),(4,1),(2,4)]
13	(2,3)	D(3,3) OBSTACLE, L(2,2) visited, U(1,3) OBSTACLE; R(2,4) already queued	— none new —	[(3,2),(4,1),(2,4)]
14	(3,2)	D(4,2) ok, L(3,1) OBSTACLE, U(2,2) visited	(4,2)	[(4,1),(2,4),(4,2)]
15	(4,1)	L(4,0) visited, U(3,1) OBSTACLE; R(4,2) already queued	— none new —	[(2,4),(4,2)]
16	(2,4)	D(3,4) ok, L(2,3) visited, U(1,4) visited	(3,4)	[(4,2),(3,4)]
17	(4,2)	R(4,3) ok, L(4,1) visited, U(3,2) visited	(4,3)	[(3,4),(4,3)]

18	(3,4)	D(4,4) = GOAL generated! L(3,3) OBSTACLE, U(2,4) visited	(4,4) — Goal	Goal found; search terminates

Path Reconstruction (via parent pointers)

Tracing parents backward from the Goal (4,4):

- (4,4) ← (3,4) ← (2,4) ← (1,4) ← (0,4) ← (0,3) ← (0,2) ← (0,1) ← (0,0) = S

Optimal Path: (0,0) → (0,1) → (0,2) → (0,3) → (0,4) → (1,4) → (2,4) → (3,4) → (4,4)

Path length = 8 moves → Total cost = 8 (since each move costs 1).

Optimality check: the Manhattan Distance lower bound was 8, and BFS found a path of cost exactly 8 — the path is confirmed optimal, since the achieved cost equals the lower bound.

Question 3 Solution: Online Uniform Cost Search Approach

b) Solution Approach — Uniform Cost Search as an Online Search Agent

Because path costs are not uniform (normal move = 1, risky-zone move = 3) and the map is not fully known in advance, the most suitable strategy combines Uniform Cost Search (UCS) with an Online Search Agent framework:

- Step 1 (Initialize): place S in a priority queue (frontier) ordered by cumulative path cost g(n), with g(S) = 0. Maintain a 'known map' that initially only contains the robot's current cell and its sensed neighbours.
- Step 2 (Sense): at the current cell, observe all adjacent cells; mark each as free, obstacle, or risky-zone, and add this information to the known map.
- Step 3 (Expand lowest-cost node): pop the frontier node with the smallest g(n) — standard UCS behaviour — and generate its neighbours from the known map only (unknown cells are not yet expandable).
- Step 4 (Assign edge costs): for each neighbour, add edge cost 1 if it is a normal free cell, or 3 (1 base + 2 penalty) if it is a risky zone; update g(n) = g(parent) + edge cost.
- Step 5 (Goal test): if the expanded node is G, terminate and return the path with minimum cumulative cost found so far — this is the safety/optimality guarantee UCS provides for non-negative edge costs.
- Step 6 (Move and repeat): the robot physically moves to the next node on the current best path, senses again, and if newly sensed cells invalidate the current plan, it re-runs UCS from its new position using the updated known map — this is the 'online' element, since planning and execution are interleaved rather than done once offline.
- The process repeats until the survivor's location G is physically reached.

c) Demonstration of Core AI Concepts

- Intelligent Agent: the rescue robot perceives its environment through limited local sensors (adjacent-cell observation) and acts through movement actions, continuously cycling through a sense–plan–act loop.
- Rationality: the robot behaves rationally because, given what it has perceived so far, it always selects the action expected to minimize cumulative cost while guaranteeing it does not knowingly move into an obstacle.
- Environment type: partially observable (only adjacent cells sensed), deterministic (moves have certain outcomes), sequential (each move affects future options), discrete (grid of rooms, discrete actions), and single-agent.

d) Justification of the Chosen Approach

- Correct cost model: unlike plain BFS, UCS expands nodes in order of true cumulative path cost g(n), essential since risky-zone moves cost more than normal moves.
- Optimality guarantee: for non-negative edge costs (1 or 3 here), UCS is guaranteed to find the minimum-cost path once the goal is reachable.
- Compatibility with partial observability: UCS only needs a priority queue and a cost function, so it naturally extends into an online agent that re-searches whenever new sensing information changes the known map.
- Safety: obstacles are only added to the known map once sensed, and UCS never expands into unconfirmed cells, so the robot never risks planning through a blocked cell.

Conclusion: an Online Uniform Cost Search agent is better suited to this rescue-robot scenario than either plain BFS (which ignores variable move costs) or a fully offline search (which assumes complete map knowledge the robot does not have).